PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Nome do Projeto:	SIGA UEA – Sistema de Gestão de TCC do NUCOMP-UEA
Instituição:	UEA – Universidade do Estado do Amazonas, Escola Superior de Tecnologia (EST) – NUCOMP
Patrocinadora / Cliente:	Profª Drª Diana Maria da Câmara – Coordenação do NUCOMP
Equipe / Gerentes do Projeto:	Davih Taumaturgo Soares, Igor Leite de Almeida, Igor Meireles de Lima Taveira, Matheus Maia Braga e Samuel Ogino Barbosa.
Documento de referência:	Plano de Integração do Projeto, versão 2.6, seção 4.6.3
Data:	23/10/2026
Versão:	1.3

SUMÁRIO

1. INTRODUÇÃO.....3
1.1 Objetivo.....3
1.2 Escopo.....3
1.3 Documentos de referência.....3
1.4 Glossário.....4
2. ITENS DE CONFIGURAÇÃO.....4
3. REPOSITÓRIO E ESTRATÉGIA DE BRANCHES.....6
3.1 Estrutura do repositório.....6
3.2 Modelo de branches.....6
3.3 Fluxo de uma tarefa.....7
4. COMMITS, PULL REQUESTS E REVISÃO.....7
4.1 Commits.....7
4.2 Pull requests.....7
4.3 Revisores.....8
5. VERSIONAMENTO E BASELINES.....8
6. CONTROLE DE MUDANÇAS.....9
6.1 Quando abrir uma solicitação.....9
6.2 Fluxo.....9
6.3 Regras específicas.....10
7. AMBIENTES, BACKUP E AUDITORIA.....10
7.1 Ambientes.....10
7.2 Configuração e segredos.....10
7.3 Backup.....11
7.4 Auditoria da configuração.....11
8. PAPÉIS E RESPONSABILIDADES.....11
9. PENDÊNCIAS E DECISÕES EM ABERTO.....11
10. REGISTRO DE ALTERAÇÕES.....12
11. APROVAÇÕES.....12
12. REFERÊNCIAS.....13

1. INTRODUÇÃO

Este documento apresenta o Plano de Gerenciamento de Configuração do SIGA UEA – Sistema de Gestão de TCC do NUCOMP-UEA. Ele define como o código, o banco de dados e os documentos do projeto são versionados, revisados, liberados e alterados, e detalha a seção 4.6.3 (gestão de configuração) do Plano de Integração do Projeto. As regras descrevem o repositório `SIGA-UEA/sigauea` como ele está em 23/10/2026 e indicam, quando for o caso, o que ainda precisa ser ajustado nele.

1.1 Objetivo

Garantir que cada artefato do projeto tenha histórico, responsável, revisor e versão identificada, e que toda mudança em item já entregue seja registrada, analisada e comunicada à equipe.

1.2 Escopo

O plano vale para os cinco integrantes da equipe, do início da modelagem até o encerramento do projeto, em 09/12/2026, e cobre o código-fonte, o esquema do banco de dados e suas migrações, os documentos de requisitos, de modelagem e de gestão, os protótipos de telas e a configuração dos ambientes.

1.3 Documentos de referência

- Plano de Integração do Projeto – SIGA UEA, versão 2.6, de 23/10/2026, em especial as seções 4.6.3 (gestão de configuração) e 4.7 (controle de mudanças).
- Requisitos_SIGA_UEA.xlsx: requisitos de alto e baixo nível, regras de negócio, requisitos não funcionais, fluxos, telas e rastreabilidade.
- README.md do repositório, `render.yaml`, `docker-compose.yml` e `backend/.env.exemplo`, que descrevem a execução e a implantação do sistema.

1.4 Glossário

Termo	Significado
Baseline	Conjunto de itens de configuração e de versões congelado em um marco ou tag.
Branch	Ramificação do repositório Git: main, dev, feature, fix e hotfix.
CCM	Comitê de Controle de Mudanças.
Commit	Registro de uma alteração no repositório, no formato tipo(ID): frase.
CR	Change Request, ou solicitação de mudança, numerada no formato CR-2026-NNNN.
CT	Caso de teste da matriz de testes (`docs/matriz-ct-testes.md`).
Git / GitHub	Controle de versão do código e plataforma que hospeda o repositório SIGA-UEA/sigauea.
Item de configuração	Elemento do projeto sob controle de versão e de mudanças (código, banco, documentos).
Migração (Alembic)	Script versionado que cria ou altera o esquema do banco de dados.
FSM	Formulário de Solicitação de Mudança.
PR	Pull request: pedido de integração de código, com revisão por outro integrante.
Tag	Marca imutável no Git que identifica uma versão entregue, no padrão vMAIOR.MENOR.CORREÇÃO.
RCM	Relatório de Controle de Mudanças.
RAN e RBN	Requisitos de alto e de baixo nível da planilha de requisitos.
TI	Teste de integração registrado nos cards do JIRA.
Entrega 1 a 4	Entregas ao cliente, em 30/09, 25/10, 18/11 e 09/12/2026.

2. ITENS DE CONFIGURAÇÃO

Item de configuração é todo artefato que muda ao longo do projeto e precisa de histórico, revisão e versão identificada. A tabela a seguir lista os itens, o local de cada um no repositório e quem responde por ele. Os itens marcados com “mover” estão hoje na raiz do repositório, e os marcados com “a criar” ainda não têm pasta (seção 9).

Item de configuração	Formato	Local no repositório	Responsável	Revisor
Requisitos (RAN, RBN, RN, RNF, fluxos e telas)	Planilha .xlsx	Raiz (`Requisitos_SIGA_UEA (4).xlsx`); destino `docs/requisitos` (mover)	Igor Meireles	Davih Taumaturgo
Plano de Integração, EAP e este plano	.docx, .pdf e .drawio	`docs/gestao` (a criar)	Igor Meireles	Davih Taumaturgo

Item de configuração	Formato	Local no repositório	Responsável	Revisor
Protótipos de telas	HTML	Raiz (`SIGA UEA — Protótipo.html`); destino `docs/prototipos` (mover)	Igor Leite	Matheus Maia
Histórias de usuário, diagramas de sequência, de classes, de componentes, de pacotes e de implantação	.docx e .drawio	`docs/modelagem` e `docs/arquitetura`; diagramas de sequência na raiz (`SIGA_UEA_FL01-10_diagramasSeq.drawio`), a mover	Igor Meireles	Davih Taumaturgo
Modelo de dados (DER, modelo lógico e script SQL)	.drawio, .md e .sql	`docs/modelagem`, `docs/modelo-de-dados.md` e `docs/esquema-postgresql.sql`	Davih Taumaturgo	Samuel Ogino
Migrações do banco	Alembic (.py)	`backend/migrations/versions`	Davih Taumaturgo	Samuel Ogino
Contrato de API	OpenAPI e documento	`docs/openapi.json` e `docs/arquitetura/Contrato_da_API_SIGA_UEA`	Igor Meireles	Igor Leite e Davih Taumaturgo
Código do frontend (Next.js, React, TypeScript e Tailwind)	Código-fonte	Raiz: `app/`, `components/`, `lib/`, `proxy.ts` e `next.config.ts`	Igor Leite	Matheus Maia
Código do backend (Python e FastAPI)	Código-fonte	`backend/app`	Davih Taumaturgo	Samuel Ogino
Serviço de e-mail e relatórios	Código-fonte	`backend/app/services` (e-mail) e módulo de relatórios, quando existir	Samuel Ogino	Igor Meireles
Testes (pytest, e2e, carga, matriz de CT e TI)	Código e planilhas	`backend/tests`, `backend/scripts`, `docs/testes` e `docs/matriz-ct-testes.md`	Igor Meireles	Integrante que não escreveu o teste
Implantação e ambientes (Docker, Render, Vercel e variáveis)	Configuração	Raiz: `Dockerfile`, `docker-compose.yml`, `render.yaml` e `.vercelignore`; `backend/Dockerfile` e `backend/.env.exemplo`	Igor Leite	Matheus Maia

O quadro de tarefas fica no JIRA, e a planilha de Planning Poker, o cronograma e os entregáveis da Entrega 1 ficam fora do repositório (pasta de entregáveis do TP1). Os cards do JIRA registram os identificadores dos requisitos e as evidências, conforme a Definição de Pronto do Plano de Integração (4.1.5).

3. REPOSITÓRIO E ESTRATÉGIA DE BRANCHES

3.1 Estrutura do repositório

O projeto utiliza um repositório Git único, `SIGA-UEA/sigauea`, no GitHub, em plano gratuito. O site Next.js fica na raiz e a API fica em `backend`. A estrutura atual é a seguinte.

```
sigauea/
  app/  components/  lib/     site Next.js (telas e chamadas à API)
  proxy.ts  next.config.ts   login e repasse de /api/v1 à API
  backend/
    app/            API FastAPI: routes, services, schemas, models.py
    migrations/     migrações Alembic (versions/0001 a 0004)
    tests/          testes pytest, um por critério de aceite
    scripts/        teste de carga (RNF01)
    .env.exemplo    variáveis de ambiente, sem segredos
  docs/
    arquitetura/    arquitetura, contrato de API e diagramas de implantação
    modelagem/      DER, modelo lógico, diagrama de implantação
    testes/         relatório de execução, e2e e evidências
    openapi.json  esquema-postgresql.sql  modelo-de-dados.md  matriz-ct-testes.md
  Dockerfile  docker-compose.yml  render.yaml  .vercelignore
  README.md
```

Não existem as pastas `frontend`, `database` e `infra`: o frontend ocupa a raiz, o esquema do banco vem das migrações em `backend/migrations`, e a configuração de implantação fica na raiz e em `backend`.

3.2 Modelo de branches

Adota-se o modelo GitFlow simplificado: a branch `main` guarda somente versões entregues, a branch `dev` recebe a integração contínua e cada tarefa possui a sua própria branch.

Branch	Origem	Destino	Uso	Proteção
`main`	—	—	Versões entregues (marcos e entregas), cada uma com tag. Publicada automaticamente no ambiente integrado (seção 7.1).	Protegida: recebe apenas PR vindo da `dev`.
`dev`	`main`	`main`	Integração do código de todas as tarefas.	Protegida: recebe apenas PR revisado.
`feature/<ID>-<descrição>`	`dev`	`dev`	Uma tarefa ou requisito. Exemplo: `feature/RBN03.01-formulario-cadastro-tcc`.	Removida após a integração.
`fix/<ID>-<descrição>`	`dev`	`dev`	Correção de defeito encontrado nos testes.	Removida após a integração.
`hotfix/<descrição>`	`main`	`main` e `dev`	Correção urgente em versão já entregue.	Removida após a integração.

3.3 Fluxo de uma tarefa

1. O integrante cria a branch `feature/` a partir da `dev` atualizada.
2. Desenvolve em commits pequenos, cada um identificado pelo requisito correspondente (seção 4.1).
3. Abre um pull request para a `dev` e solicita revisão ao revisor definido na seção 4.3.
4. Após a aprovação do revisor e dos testes, o PR é integrado e a branch é removida.
5. A cada marco de integração, a `dev` é integrada à `main` por PR, e a `main` recebe a tag da versão.

Caso a proteção automática de branch não esteja disponível no plano gratuito do GitHub, a regra permanece válida como regra da equipe: nenhum integrante realiza commit direto na `main` ou na `dev`.

4. COMMITS, PULL REQUESTS E REVISÃO

4.1 Commits

A mensagem de cada commit segue o formato `tipo(ID): frase curta no imperativo`, em que o tipo é `feat` (funcionalidade), `fix` (correção), `test` (testes), `docs` (documentos) ou `infra` (ambiente e implantação), e o ID é o do requisito, da regra ou do caso de teste. Exemplos: `feat(RBN03.01): valida campos obrigatórios do cadastro de TCC`, `fix(RN04): bloqueia segundo cadastro no mesmo período` e `test(CT-HT07-01): adiciona caso de teste de agenda`. Quando não houver requisito, o ID pode ser omitido (`infra: ajusta o health check do Render`). Uma mudança aprovada por solicitação de mudança cita a CR no corpo da mensagem.

4.2 Pull requests

O título do PR repete o tipo e o identificador do requisito, e a descrição indica o card do JIRA e o que foi testado. O PR somente é integrado quando cumpre todos os critérios abaixo, derivados da Definição de Pronto do Plano de Integração.

- Atende aos RBN e aos RN e RNF aplicáveis.
- Foi aprovado nos testes previstos (`python -m pytest` no backend e os testes de integração do requisito).
- Foi aprovado por outro integrante, que não é o autor.
- Traz a migração Alembic correspondente quando altera `backend/app/models.py`; o teste `test_migracoes_batem_com_os_modelos` deve passar.
- Não altera contrato de API, esquema do banco ou requisito sem solicitação de mudança aprovada (seção 6).
- Não contém senhas, chaves de acesso, arquivos `.env` ou dados reais de usuários.

A integração do PR é feita por squash, de modo que a `dev` mantenha um commit por tarefa.

4.3 Revisores

Os revisores seguem a tabela de equipe do Plano de Integração (seção 4.3.1).

Autor	Revisor
Davih Taumaturgo Soares	Samuel Ogino Barbosa
Igor Leite de Almeida	Matheus Maia Braga
Igor Meireles de Lima Taveira	Davih Taumaturgo Soares
Matheus Maia Braga	Igor Leite de Almeida
Samuel Ogino Barbosa	Igor Meireles de Lima Taveira

Propõe-se que cada revisão ocorra em até dois dias úteis. O atraso é tratado como impedimento na reunião diária.

5. VERSIONAMENTO E BASELINES

As versões seguem o padrão `vMAIOR.MENOR.CORREÇÃO`. Enquanto o sistema não está completo, o número maior permanece em 0, e a versão 1.0.0 marca a entrega final. Cada marco a partir da Entrega 2 (25/10/2026) recebe uma tag na `main`; antes dela não há tags. Baseline é o conjunto de itens de configuração congelado em determinada data ou tag. As datas e os conteúdos abaixo são os do Plano de Integração (seção 4.2.3).

Data	Tag	Marco	Baseline (itens congelados)
30/09/2026	sem tag	Entrega 1	EAP, Planning Poker, Plano de Integração, requisitos, cronograma e telas, na pasta de entregáveis do TP1.
14/10/2026	sem tag	Marco de integração 1	Banco, módulo de usuários e contrato de API integrados.
25/10/2026	`v0.2.0`	Marco de integração 2 e Entrega 2	Documentação do sistema, modelo de dados lógico e físico, regras lógicas implementadas, este plano e cadastro de TCC de ponta a ponta em ambiente integrado.
06/11/2026	`v0.3.0`	Marco de integração 3	Análise do cadastro pelo coordenador e e-mail ao orientador integrados.
18/11/2026	`v0.4.0`	Entrega 3	Acompanhamento, dashboard e ideias entregues.
02/12/2026	`v0.5.0`	Marco de integração 4	Defesa, publicação, dashboard e agenda integrados; ideias conforme a versão.
09/12/2026	`v1.0.0`	Entrega 4 e encerramento	Sistema completo, documentação técnica e manual de uso.

A Entrega 1 e o Marco 1 ocorreram antes da adoção deste plano e não receberam tag no repositório. Elas ficam registradas como baselines históricas, e a primeira tag é a `v0.2.0`.

Os documentos externos ao código mantêm seu próprio histórico de versão. A mensagem de cada tag anota a versão exata de cada documento, para que a baseline aponte para eles. Para a `v0.2.0`, a mensagem deve registrar:

Documento	Versão	Observação
Plano de Integração do Projeto	2.6	23/10/2026.
Plano de Gerenciamento de Configuração	1.3	Este documento.
Documento de Sequência	0.9	23/10/2026, inclui o diagrama de classes por módulo.
Histórias de Usuário (sprints S01 a S03)	1.1	Aprovação formal não realizada.
EAP do SIGA UEA	1.0	Sem alteração.
Arquitetura do Sistema e Contrato da API	1.0	Em `docs/arquitetura`; citam versões anteriores deste plano e do Plano de Integração (seção 9).
Requisitos_SIGA_UEA.xlsx	a registrar	A equipe deve anotar a versão antes da tag.

Depois de criada a tag, um item congelado somente pode ser alterado por solicitação de mudança aprovada (seção 6), e a alteração segue para a versão seguinte. A tag é criada com `git tag -a v0.2.0 -m "<mensagem>"` e enviada com `git push origin v0.2.0`. As datas acompanham o cronograma do Plano de Integração e podem mudar com ele.

6. CONTROLE DE MUDANÇAS

O controle de mudanças utiliza os modelos já existentes no Plano de Integração: o Formulário de Solicitação de Mudança (FSM, seção 4.7.2) e o Relatório de Controle de Mudanças (RCM, seção 4.7.3). Cada solicitação recebe um número no formato `CR-2026-NNNN`, como a CR-2026-0003 registrada na seção 4.7.2.

6.1 Quando abrir uma solicitação

A solicitação é obrigatória para mudança de escopo, prazo, custo ou interfaces, como o contrato de API, o esquema do banco ou um requisito pertencente a uma baseline. A correção de defeito e o ajuste de código que não altere requisito ou interface seguem apenas o fluxo de pull request.

6.2 Fluxo

1. O solicitante preenche o FSM com descrição, justificativa e análise de impacto em escopo, cronograma, custo, qualidade e integração.
2. O Comitê de Controle de Mudanças (CCM) analisa a solicitação e registra o parecer.
3. Se aprovada, a mudança é registrada no RCM, e o item afetado é atualizado com o número da CR no corpo da mensagem do commit.
4. Os vínculos de rastreabilidade (RAN, RBN, RN, RNF, telas e testes) são atualizados na planilha de requisitos.
5. Se reprovada, a decisão e a justificativa ficam registradas no RCM.

6.3 Regras específicas

- **Contrato de API.** A alteração de rota, formato ou código de erro exige comunicação prévia ao frontend e ao backend e o registro de uma CR, conforme o risco R02 do Plano de Integração. Aprovada a mudança, o arquivo `docs/openapi.json` e o documento do contrato são regenerados no mesmo PR.
- **Esquema do banco.** Toda alteração deve ser feita por migração Alembic numerada em `backend/migrations/versions` (a próxima é a `0005`), criada com `alembic revision --autogenerate`, e nunca por alteração manual no banco do ambiente integrado. O modelo de dados e o script em `docs/` são atualizados no mesmo PR.
- **Requisito de uma baseline.** Depois da tag, a alteração de um requisito exige CR e atualização da planilha de requisitos, das histórias no JIRA e dos testes relacionados.

7. AMBIENTES, BACKUP E AUDITORIA

7.1 Ambientes

Ambiente	Finalidade	Usuários	Atualização
Local	Desenvolvimento e testes de cada integrante. Pode usar SQLite (padrão, sem configuração), com `uvicorn --reload` e `npm run dev`, ou Docker Compose com PostgreSQL 16, API e site.	Cada integrante	Pelo próprio integrante, a partir da `dev`.
Integrado e homologação	Testes de integração de sistema, testes de RNF e validação com a coordenação. Site na Vercel, API no Render e banco no Neon, com HTTPS.	Toda a equipe	Automática a cada integração na `main` (`autoDeploy` do Render e Production Branch da Vercel). Para testar a `dev` antes de um marco, troca-se a branch nas duas plataformas, e a troca é registrada no relatório de status.

O responsável pela implantação e pela configuração dos ambientes é Igor Leite. A conclusão do ambiente (HTTPS, deploy e e-mail) é acompanhada na história SCRUM-111 do JIRA. O Render grátis bloqueia SMTP, e por isso o e-mail do ambiente integrado sai pela API da Brevo (`BREVO_API_KEY`); no ambiente local, o envio usa SMTP ou aparece no terminal da API.

7.2 Configuração e segredos

Senhas, chaves e dados de acesso ficam em variáveis de ambiente, fora do repositório. O `.gitignore` exclui os arquivos `.env*`, exceto `.env.exemplo`, que contém os nomes das variáveis (`backend/.env.exemplo`) e valores fictícios. No Render, as variáveis sensíveis (`DATABASE_URL`, `SMTP_SENHA`, `BREVO_API_KEY` e outras marcadas com `sync: false` no `render.yaml`) são informadas na criação do serviço, e na Vercel a variável `API_URL` é configurada no painel. O comando `python -m app.seed --demo` cria contas de demonstração com senha conhecida e nunca deve ser executado no ambiente integrado. Os dados de teste não devem ser dados reais de alunos ou professores.

7.3 Backup

O código e os documentos ficam no GitHub, além das cópias locais de cada integrante. O plano gratuito do Neon mantém apenas 6 horas de histórico para restauração (NEON, 2026), o que não cobre uma falha percebida no dia seguinte. Por isso, propõe-se que Igor Leite faça um dump diário do banco do ambiente integrado com `pg_dump`, guardando os últimos sete fora do repositório, e um dump adicional antes de cada marco de integração.

7.4 Auditoria da configuração

Antes de cada tag, o Scrum Master confere se os pull requests integrados possuem revisor, se as mudanças de requisito possuem CR, se a planilha de requisitos está alinhada ao código, se a suíte de testes passa (incluindo o teste de migrações) e se nenhum arquivo `.env` foi versionado. O resultado é registrado no relatório de status da sprint. Essa auditoria do processo é distinta do log de auditoria do sistema (RNF03).

8. PAPÉIS E RESPONSABILIDADES

Papel	Responsável	Responsabilidade na configuração
Gerente de configuração	Igor Meireles	Guarda as baselines, cria as tags, abre e acompanha as CRs, configura a proteção de branch e audita antes de cada tag.
Banco e migrações	Davih Taumaturgo	Mantém as migrações Alembic, o modelo de dados e os scripts em `docs/`.
Frontend, implantação e backup	Igor Leite	Mantém o site Next.js e os protótipos; responde pelos ambientes, pela implantação (Render, Vercel e Neon) e pelo dump do banco.
Comunicação	Matheus Maia	Registra as decisões e as CRs comunicadas ao cliente e aos interessados.
E-mail e relatórios	Samuel Ogino	Mantém o serviço de e-mail (SMTP e Brevo) e os relatórios, e revisa o código do backend.

A distribuição de papéis é uma proposta baseada nas áreas do Plano de Integração e deve ser confirmada pela equipe. A responsabilidade pela implantação e pelo backup, hoje com Igor Leite, difere da área “Banco de dados e hospedagem” do RACI (item 3, Davih Taumaturgo); a equipe deve confirmar qual vale.

9. PENDÊNCIAS E DECISÕES EM ABERTO

- **Arquivos soltos na raiz.** Requisitos (`Requisitos_SIGA_UEA (4).xlsx`, com a versão no nome do arquivo), protótipo HTML, PDF das histórias de usuário e diagramas de sequência devem ir para as pastas `docs/requisitos`, `docs/prototipos`, `docs/gestao` e `docs/modelagem`, e as pastas `docs/gestao`, `docs/requisitos` e `docs/prototipos` precisam ser criadas.
- **Commits diretos na `dev`.** No clone verificado em 23/10/2026, os commits recentes da `dev` foram feitos diretamente nela. A regra de PR por tarefa e de revisão por outro integrante vale a partir desta versão.

- **Proteção de branch.** Depende dos recursos do plano gratuito do GitHub para o tipo de repositório adotado.
- **Composição do CCM.** O Plano de Integração não define os integrantes do Comitê de Controle de Mudanças.
- **Valores propostos.** O prazo de revisão de dois dias úteis, o dump diário e a retenção de sete cópias são propostas desta versão, sem base no Plano de Integração.
- **Responsável pela implantação e pelo backup.** Igor Leite, conforme decisão da equipe em 23/10/2026; o RACI do Plano de Integração atribui hospedagem a Davih Taumaturgo (seção 8).
- **Documentos que citam versões anteriores.** O documento de Arquitetura do Sistema, em `docs/arquitetura`, cita o Plano de Integração v2.3 e este plano na versão 1.1, e deve ser atualizado na próxima emissão.
- **Homologação a partir da `main`.** Como o ambiente integrado publica a `main`, o teste de integração de uma funcionalidade ainda na `dev` exige trocar a branch nas plataformas; a equipe deve decidir se mantém essa regra ou cria um ambiente de homologação próprio para a `dev`.

10. REGISTRO DE ALTERAÇÕES

Versão	Data	Descrição	Responsável
1.0	11/10/2026	Criação do documento.	Igor Meireles
1.1	20/10/2026	Atualização de datas, ferramentas e referências ao Plano de Integração v2.3; padronização da linguagem.	Igor Meireles
1.2	23/10/2026	Alinhamento ao repositório real e ao Plano de Integração v2.5: estrutura de pastas, branch `dev`, migrações Alembic, commits no formato tipo(ID), ambientes local e integrado, marcos e baselines do Plano de Integração, tags a partir da v0.2.0, responsável pela implantação e pelo backup e versões dos documentos da baseline.	Igor Meireles
1.3	23/10/2026	Reprogramação da Entrega 2 para 25/10/2026 (CR-2026-0004 do Plano de Integração v2.6): primeira tag `v0.2.0` em 25/10, marco 3 em 06/11, glossário e versões dos documentos da baseline atualizadas.	Igor Meireles

11. APROVAÇÕES

Os campos abaixo se destinam à assinatura de quem aprova este plano. Enquanto não houver aprovação, permanecem em branco.

Assinatura – Equipe de desenvolvimento	Assinatura – Profª Drª Diana Maria da Câmara / Coordenação NUCOMP	Data

12. REFERÊNCIAS

- SIGA UEA. Plano de Integração do Projeto. Versão 2.6. Manaus: NUCOMP – EST/UEA, 23 out. 2026.
- SIGA UEA. Requisitos_SIGA_UEA.xlsx: matriz de requisitos, regras de negócio, requisitos não funcionais, fluxos, telas e rastreabilidade. Manaus: NUCOMP – EST/UEA, 2026.
- SIGA UEA. README.md do repositório SIGA-UEA/sigauea. GitHub, 2026.
- NEON. History window. Disponível em: https://neon.com/docs/postgres/backup-restore/history-window. Acesso em: 23 out. 2026.